



# Prelims

- ▼ Define IOT & Discuss Evolution of IOT.

## **Definition of IoT (Internet of Things):**

IoT is a network of physical devices (things) embedded with sensors, software, and internet connectivity that collect and exchange data automatically.

---

## **Evolution of IoT (Brief Points):**

1. **1980s–1990s:** Early connected devices (e.g., smart vending machine).
2. **1999:** Term “IoT” coined by Kevin Ashton.
3. **2000–2010:** Growth of internet, Wi-Fi, Bluetooth, RFID; M2M communication.
4. **2010–2015:** Rise of smartphones, cloud computing, and smart devices.
5. **2015–2020:** Integration with cloud, big data, and AI; expansion in industries.
6. **2020–Present:** Use of 5G, edge computing, smart cities, and automation.

- ▼ List four communication models used in IOT and explain any two with diagram.

## **Four Communication Models used in IoT:**

1. Device-to-Device (D2D)
2. Device-to-Cloud (D2C)
3. Device-to-Gateway (D2G)
4. Back-End Data Sharing

---

## **1. Device-to-Device (D2D)**

- Devices communicate directly with each other without cloud.

- Uses technologies like Bluetooth, Zigbee, etc.

**Diagram:**

```
[Device A] <-----> [Device B]
```

**Explanation:**

- Data is exchanged directly between devices.
- Fast and low latency.
- Example: Smart bulb controlled by a switch.

## 2. Device-to-Cloud (D2C)

- Devices send data directly to a cloud server.

**Diagram:**

```
[Device] -----> [Cloud Server] -----> [User App]
```

**Explanation:**

- Device connects to cloud via internet (Wi-Fi/4G/5G).
- Cloud stores and processes data.
- User can monitor/control remotely.
- Example: Smart thermostat connected to mobile app.

▼ List Challenges of IOT.

**Challenges of IoT (Brief Points):**

1. **Security & Privacy:** Risk of hacking and data breaches.
2. **Data Management:** Huge amount of data storage and processing.
3. **Connectivity Issues:** Reliable internet required at all times.
4. **Interoperability:** Different devices and standards may not work together.
5. **Scalability:** Managing large number of devices is difficult.
6. **Power Consumption:** Many devices depend on battery life.

7. **Cost:** High initial setup and maintenance cost.

8. **Complexity:** System design and integration can be complicated.

▼ Explain Characteristics of IOT

**Characteristics of IoT (Brief Points):**

1. **Connectivity:** Devices are connected to the internet for communication.

2. **Sensing:** Uses sensors to collect data from the environment.

3. **Intelligence:** Data is processed to make smart decisions.

4. **Automation:** Devices can operate without human intervention.

5. **Heterogeneity:** Different types of devices and technologies work together.

6. **Scalability:** Can handle a large number of connected devices.

7. **Dynamic Changes:** Devices can join/leave and change states frequently.

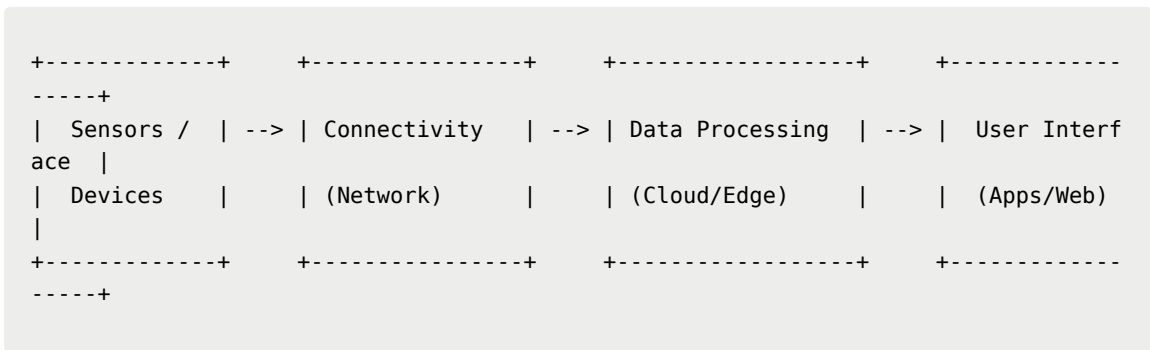
8. **Security:** Requires protection of data and devices.

▼ Differentiate between REST API and web socket API.

| Feature                   | REST API                          | WebSocket API                         |
|---------------------------|-----------------------------------|---------------------------------------|
| <b>Communication Type</b> | Request-Response                  | Full-duplex (two-way)                 |
| <b>Connection</b>         | Stateless (new request each time) | Persistent connection                 |
| <b>Protocol</b>           | Uses HTTP/HTTPS                   | Uses WebSocket (ws/wss)               |
| <b>Data Flow</b>          | Client must request data          | Server can push data anytime          |
| <b>Speed</b>              | Slower (overhead per request)     | Faster (low latency after connection) |
| <b>Use Case</b>           | CRUD operations, web services     | Real-time apps (chat, gaming)         |
| <b>Complexity</b>         | Simple to implement               | More complex to manage                |

▼ Draw block diagram of IOT and explain function of each diagram.

**Block Diagram of IoT:**



# Functions of Each Block:

## 1. Sensors / Devices

- Collect data from the environment (temperature, motion, humidity, etc.)
- Actuators can also perform actions (e.g., switch ON/OFF devices)

## 2. Connectivity (Network)

- Transfers data from devices to processing systems
- Uses Wi-Fi, Bluetooth, cellular networks, etc.

## 3. Data Processing (Cloud/Edge)

- Stores, analyzes, and processes the collected data
- Makes decisions using algorithms or AI

## 4. User Interface (Application Layer)

- Displays data to users via mobile apps or web dashboards
- Allows users to control IoT devices remotely

▼ Define sensor and transducer. Explain the relationship between them with examples.

### Sensor:

A sensor is a device that detects or measures a physical quantity (like temperature, light, pressure) and responds to it.

### Transducer:

A transducer is a device that converts one form of energy into another (usually physical quantity into electrical signal).

---

## Relationship between Sensor and Transducer:

- A **sensor** is a **type of transducer**.
  - Specifically, it is an **input transducer** that converts physical parameters into electrical signals.
  - All sensors are transducers, but not all transducers are sensors (some are output devices like speakers).
- 

## Examples:

### 1. Temperature Measurement

- Sensor: Thermistor
- Transducer action: Converts temperature → electrical resistance

### 2. Microphone

- Sensor: Detects sound waves
- Transducer action: Converts sound → electrical signal

### 3. LDR (Light Dependent Resistor)

- Sensor: Detects light intensity
- Transducer action: Converts light → change in resistance

### 4. Speaker (Output Transducer)

- Not a sensor
- Converts electrical signal → sound

▼ Compare Raspberry Pi and Arduino (any 8 points).

| Feature                    | Raspberry Pi             | Arduino                            |
|----------------------------|--------------------------|------------------------------------|
| <b>1. Type</b>             | Single-board computer    | Microcontroller                    |
| <b>2. Operating System</b> | Runs OS (Linux)          | No OS                              |
| <b>3. Processor</b>        | Powerful CPU (GHz range) | Simple microcontroller (MHz range) |

| Feature                     | Raspberry Pi                        | Arduino                                   |
|-----------------------------|-------------------------------------|-------------------------------------------|
| <b>4. RAM</b>               | High (GBs)                          | Very low (KBs)                            |
| <b>5. Programming</b>       | Python, C++, Java, etc.             | Mainly C/C++                              |
| <b>6. Boot Time</b>         | Takes time to boot                  | Starts instantly                          |
| <b>7. Real-time Control</b> | Not suitable for real-time          | Suitable for real-time tasks              |
| <b>8. Applications</b>      | Complex tasks (web server, IoT, AI) | Simple hardware control (sensors, motors) |

▼ Explain any five sensors used in Home Automation.

### **Five Sensors used in Home Automation (Brief Explanation):**

#### **1. Temperature Sensor**

- Measures room temperature.
- Used in smart ACs and thermostats for automatic temperature control.

#### **2. Motion Sensor (PIR Sensor)**

- Detects movement of people.
- Used for automatic lights, security alarms.

#### **3. Light Sensor (LDR)**

- Detects light intensity.
- Used to turn lights ON/OFF automatically based on daylight.

#### **4. Gas Sensor**

- Detects harmful gases (like LPG, smoke).
- Used for safety and fire prevention systems.

#### **5. Humidity Sensor**

- Measures moisture level in air.
- Used in smart fans, ACs, and ventilation systems.

▼ Discuss advantages & Limitation of actuators in IoT System.

### **Actuators in IoT:**

Devices that convert electrical signals into physical actions (e.g., motor movement, switching, valve control).

---

## Advantages of Actuators in IoT:

1. **Automation:** Enable automatic control of devices (lights, doors, appliances).
  2. **Remote Control:** Devices can be operated from anywhere via internet.
  3. **Efficiency:** Improves energy and resource utilization.
  4. **Real-time Action:** Immediate response to sensor data.
  5. **Reduced Human Effort:** Minimizes manual intervention.
- 

## Limitations of Actuators in IoT:

1. **Power Consumption:** Many actuators require high power.
  2. **Cost:** Installation and maintenance can be expensive.
  3. **Complexity:** Integration with IoT systems can be difficult.
  4. **Security Risks:** Vulnerable to hacking if not secured properly.
  5. **Reliability Issues:** Failure can affect entire system operation.
- 

### In short:

Actuators enable action in IoT systems but come with cost, power, and security challenges 👍

- ▼ Discuss static and dynamic characteristics of sensors with examples.

## 1. Static Characteristics

(Performance when input is constant)

- **Accuracy:** Closeness of measured value to true value *Example:* A temperature sensor showing 25°C when actual is 25°C
- **Precision:** Consistency of repeated measurements *Example:* Sensor repeatedly shows 24.9°C
- **Sensitivity:** Change in output per unit input change *Example:* Small temperature change causes noticeable output change

- **Linearity:** Output proportional to input *Example:* Voltage increases linearly with temperature
  - **Range:** Minimum to maximum measurable values *Example:* 0°C to 100°C
- 

## 2. Dynamic Characteristics

(Performance when input changes with time)

- **Response Time:** Time taken to respond to input change *Example:* Sensor takes 2 seconds to show new temperature
  - **Speed of Response:** How quickly output reaches final value
  - **Lag:** Delay between input and output
  - **Fidelity:** Ability to reproduce input signal accurately
  - **Dynamic Error:** Difference during changing conditions
- 

In short:

- **Static** → **steady conditions**
- **Dynamic** → **changing conditions** 👍

▼ What is Arduino? Explain its importance in IoT development.

**Arduino — Definition:**

**Arduino** is an open-source microcontroller-based platform used to build electronic projects. It consists of hardware (Arduino board) and software (Arduino IDE) to program and control devices.

---

## Importance of Arduino in IoT Development:

### 1. Easy to Use:

- Simple programming (C/C++) and beginner-friendly platform.

### 2. Low Cost:

- Affordable boards make IoT development accessible.

### 3. Hardware Interfacing:

- Easily connects with sensors and actuators.

### 4. Real-time Control:

- Suitable for real-time data collection and control systems.

#### 5. **Open Source:**

- Large community support, libraries, and resources available.

#### 6. **Flexibility:**

- Can be integrated with modules like Wi-Fi (ESP8266), Bluetooth, etc.

---

#### **In short:**

Arduino is a simple, low-cost, and efficient platform widely used to build IoT systems 👍

#### ▼ Describe the architecture and working of IEEE 802.15.4.

##### **IEEE 802.15.4 — Overview:**

It is a standard for **low-rate wireless personal area networks (LR-WPANs)** used in IoT (e.g., Zigbee).

---

## **Architecture of IEEE 802.15.4:**

### **1. Device Types:**

- **Full Function Device (FFD):**
  - Can act as coordinator or router
  - Communicates with all devices
- **Reduced Function Device (RFD):**
  - Simple devices (sensors)
  - Communicates only with FFD

---

### **2. Network Topologies:**

- **Star Topology:**
    - All devices connect to a central coordinator
  - **Peer-to-Peer (Mesh):**
    - Devices communicate with each other
-

### 3. Protocol Layers:

- **Physical (PHY) Layer:**
    - Handles frequency, data rate, transmission
  - **MAC Layer:**
    - Controls channel access, addressing, error detection
- 

### Working of IEEE 802.15.4:

#### 1. Network Formation:

- Coordinator (FFD) creates the network

#### 2. Device Joining:

- Other devices (FFD/RFD) join the network

#### 3. Data Transmission:

- Devices send data using MAC protocol (CSMA/CA)

#### 4. Low Power Operation:

- Devices enter sleep mode to save energy

#### 5. Communication:

- Data flows via coordinator or directly (in mesh)
- 

#### In short:

IEEE 802.15.4 provides **low-power, low-data-rate wireless communication** for IoT devices 👍

▼ Explain 6LoWPAN and its importance in IoT.

#### 6LoWPAN — Definition:

**6LoWPAN (IPv6 over Low-Power Wireless Personal Area Networks)** is a protocol that allows **IPv6 communication over low-power, low-data-rate networks** like IEEE 802.15.4.

---

### Key Features of 6LoWPAN:

- Enables **IPv6 addressing** for small IoT devices
- Uses **header compression** to reduce packet size

- Supports **fragmentation** of large IPv6 packets
  - Works on **low-power wireless networks**
- 

## Importance of 6LoWPAN in IoT:

### 1. Direct Internet Connectivity:

- Devices can communicate over the internet using IPv6.

### 2. Large Address Space:

- Supports billions of IoT devices.

### 3. Energy Efficient:

- Designed for low-power devices.

### 4. Interoperability:

- Works with existing IP-based networks.

### 5. Scalability:

- Suitable for large-scale IoT deployments.

### 6. Flexibility:

- Used in smart homes, smart cities, industrial IoT.
- 

### In short:

6LoWPAN connects low-power IoT devices directly to the internet using IPv6 efficiently 👍

### ▼ Differentiate Zigbee, BLE, and Z-Wave protocols.

| Feature               | Zigbee                     | BLE (Bluetooth Low Energy) | Z-Wave                  |
|-----------------------|----------------------------|----------------------------|-------------------------|
| <b>Standard</b>       | Based on IEEE 802.15.4     | Part of Bluetooth standard | Proprietary protocol    |
| <b>Frequency Band</b> | 2.4 GHz (also 868/915 MHz) | 2.4 GHz                    | Sub-GHz (≈ 868/915 MHz) |
| <b>Range</b>          | Medium (10–100 m)          | Short (5–50 m)             | Medium (30–100 m)       |
| <b>Data Rate</b>      | ~250 kbps                  | Up to ~1 Mbps              | ~100 kbps               |

| Feature                  | Zigbee                      | BLE (Bluetooth Low Energy) | Z-Wave          |
|--------------------------|-----------------------------|----------------------------|-----------------|
| <b>Power Consumption</b> | Low                         | Very Low                   | Low             |
| <b>Network Topology</b>  | Mesh, Star                  | Star (mainly)              | Mesh            |
| <b>Devices Supported</b> | Large (~65,000)             | Limited                    | ~232 devices    |
| <b>Use Case</b>          | Smart homes, industrial IoT | Wearables, mobile devices  | Home automation |

- ▼ Explain IPv4 header with significance of each field

### IPv4 Header:

It is a part of an IP packet that contains control information needed for routing and delivery of data.

## IPv4 Header Format (Fields):

```
| Version | IHL | Type of Service | Total Length |
| Identification | Flags | Fragment Offset |
| Time to Live | Protocol | Header Checksum |
| Source Address |
| Destination Address |
| Options (if any) + Padding |
```

## Significance of Each Field:

### 1. Version (4 bits):

- Indicates IP version (IPv4 = 4)

### 2. IHL (Internet Header Length):

- Specifies header length

### 3. Type of Service (ToS):

- Defines priority and quality of service

### 4. Total Length:

- Size of entire packet (header + data)

## 5. Identification:

- Identifies packet fragments

## 6. Flags:

- Controls fragmentation (e.g., don't fragment)

## 7. Fragment Offset:

- Position of fragment in original packet

## 8. Time To Live (TTL):

- Limits packet lifetime (prevents looping)

## 9. Protocol:

- Specifies next-level protocol (TCP, UDP, etc.)

## 10. Header Checksum:

- Error detection for header

## 1. Source Address:

- Sender's IP address

## 1. Destination Address:

- Receiver's IP address

## 1. Options + Padding:

- Optional features + alignment

---

### In short:

IPv4 header controls how data is **addressed, routed, and delivered** across networks 👍

### ▼ Differentiate CoAP and MQTT.

| Feature                   | CoAP (Constrained Application Protocol) | MQTT (Message Queuing Telemetry Transport) |
|---------------------------|-----------------------------------------|--------------------------------------------|
| <b>Architecture</b>       | Client-Server (REST-based)              | Publish-Subscribe (broker-based)           |
| <b>Transport Protocol</b> | UDP                                     | TCP                                        |